Status	Finished
Started	Tuesday, 29 September 2026, 3:20 PM
Completed	Tuesday, 29 September 2026, 3:34 PM
Duration	13 mins 45 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

There is a malfunctioning keyboard where some letter keys do not work. All other keys on the keyboard work properly.

Given a string text of words separated by a single space (no leading or trailing spaces) and a string brokenLetters of all distinct letter keys that are broken, return the number of words in text you can fully type using this keyboard.

Example 1:

Input: text = "hello world", brokenLetters = "ad"

Output:

1

Explanation: We cannot type "world" because the 'd' key is broken.

For example:

Input	Result
hello world ad	1
Faculty Upskilling in Python Programming ak	2

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().splitlines()

    # Process lines in pairs (text, then brokenLetters)
    i = 0
    while i < len(input_data):
        if not input_data[i].strip():
            i += 1
            continue

        text = input_data[i]
        brokenLetters = input_data[i+1] if i + 1 < len(i

        # Convert broken letters to lowercase set for ur
        broken_set = set(brokenLetters.lower())
        words = text.split()
        count = 0

        for word in words:
            # Check if any character in the lowercase ve
            if not any(char in broken_set for char in wo
                count += 1

        print(count)
        i += 2

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	hello world ad	1	1	✓
✓	Welcome to REC e	1	1	✓
✓	Faculty Upskilling in Python Programming ak	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Check if a set is a subset of another set.

Example:

Sample Input1:

mango apple

mango orange

mango

output1:

yes

set3 is subset of set1 and set2

```
input2:
```

```
mango orange
```

```
banana orange
```

```
grapes
```

```
output2:
```

no

For example:

Test	Input	Result
1	mango apple mango orange mango	yes set3 is subset of set1 and set2
2	mango orange banana orange grapes	No

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input, ignoring empty
    lines = [line.strip() for line in sys.stdin if line.
    if not lines:
        return

    idx = 0
    # Check if the first line is a test case count
    if lines[idx].isdigit() and (len(lines) - 1) % 3 ==
        idx += 1

    # Process lines in chunks of 3 (set1, set2, set3)
    while idx + 2 < len(lines):
        set1 = set(lines[idx].split())
        set2 = set(lines[idx+1].split())
```

```
        set3 = set(lines[idx+2].split())

        # Check if set3 is a subset of both set1 and set
        if set3.issubset(set1) and set3.issubset(set2):
            print("yes")
            print("set3 is subset of set1 and set2")
        else:
            print("No")

        idx += 3

if __name__ == '__main__':
    solve()
```

	Test	Input	Expected	Got	
✓	1	mango apple mango orange mango	yes set3 is subset of set1 and set2	yes set3 is subset of set1 and set2	✓
✓	2	mango orange banana orange grapes	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a program to eliminate the common elements in the given 2 arrays and print only the non-repeating elements and the total number of such non-repeating elements.

Input Format:

The first line contains space-separated values, denoting the size of the two arrays in integer format respectively.

The next two lines contain the space-separated integer arrays to be compared.

Sample Input:

5 4

1 2 8 6 5

2 6 8 10

Sample Output:

1 5 10

3

Sample Input:

5 5

1 2 3 4 5

1 2 3 4 5

Sample Output:

NO SUCH ELEMENTS

For example:

Input	Result
5 4 1 2 8 6 5 2 6 8 10	1 5 10 3
5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.st
    if len(lines) < 3:
        return

    # Parse the two arrays from the second and third lines
    arr1 = list(map(int, lines[1].split()))
    arr2 = list(map(int, lines[2].split()))

    set1 = set(arr1)
    set2 = set(arr2)

    result = []
    seen = set()
```

```
    # Collect elements in arr1 that are not in arr2
    for x in arr1:
        if x not in set2 and x not in seen:
            seen.add(x)
            result.append(x)

    # Collect elements in arr2 that are not in arr1
    for x in arr2:
        if x not in set1 and x not in seen:
            seen.add(x)
            result.append(x)

    # Print results based on whether any non-repeating ele
    if not result:
        print("NO SUCH ELEMENTS")
    else:
        print(*(result))
        print(len(result))

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5 4 1 2 8 6 5 2 6 8 10	1 5 10 3	1 5 10 3	✓
✓	3 3 10 10 10 10 11 12	11 12 2	11 12 2	✓
✓	5 5 1 2 3 4 5 1 2 3 4 5	NO SUCH ELEMENTS	NO SUCH ELEMENTS	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Program to print all the distinct elements in an array. Distinct elements are nothing but the unique (non-duplicate) elements present in the given array.

Input Format:

First line take an Integer input from stdin which is array length n.

Second line take n Integers which is inputs of array.

Output Format:

Print the Distinct Elements in Array in single line which is space Separated

Example Input:

5

1 2 2 3 4

Output:

1 2 3 4

Example Input:

6

1 1 2 2 3 3

Output:

1 2 3

For example:

Input	Result
5 1 2 2 3 4	1 2 3 4

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from standard input (handles both
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # First value is the array length n
    n = int(input_data[0])

    # Extract the next n elements as the array
    arr = [int(x) for x in input_data[1 : 1 + n]]

    # dict.fromkeys() removes duplicates while strictly
    distinct_elements = list(dict.fromkeys(arr))

    # Print the distinct elements space-separated in a s
    print(*(distinct_elements))

```

```
21 if __name__ == __main__ :
22     solve()
```

	Input	Expected	Got	
✓	5 1 2 2 3 4	1 2 3 4	1 2 3 4	✓
✓	6 1 1 2 2 3 3	1 2 3	1 2 3	✓
✓	5 11 22 11 22 11	11 22	11 22	✓
✓	10 1 2 3 4 5 1 2 3 4 5	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

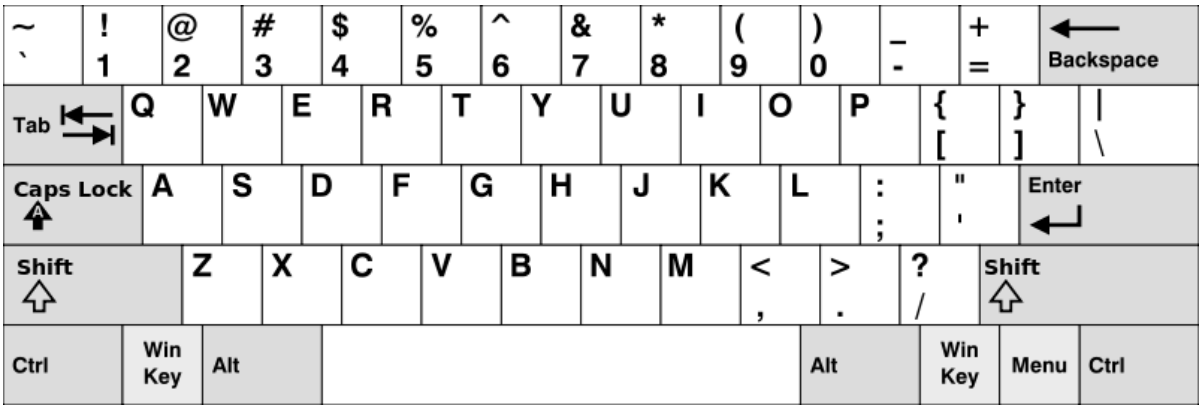
Correct

Mark 1.00 out of 1.00

Given an array of strings `words`, return *the words that can be typed using letters of the alphabet on only one row of American keyboard like the image below*.

In the **American keyboard**:

- the first row consists of the characters `"qwertyuiop"`,
- the second row consists of the characters `"asdfghjkl"`, and
- the third row consists of the characters `"zxcvbnm"`.



Example 1:

```
Input: words = ["Hello","Alaska","Dad","Peace"]
Output: ["Alaska","Dad"]
```

Example 2:

```
Input: words = ["omk"]
Output: []
```

Example 3:

```
Input: words = ["adsdf","sfd"]
Output: ["adsdf","sfd"]
```

For example:

Input	Result
4 Hello Alaska Dad Peace	Alaska Dad
2 adsfd afd	adsfd afd

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from standard input
    input_data = sys.stdin.read().split()
```

```
    if not input_data:
        return

    # First token is the number of words n
    n = int(input_data[0])
    words = input_data[1 : 1 + n]

    # Define the character sets for each keyboard row (low
    r1 = set("qwertyuiop")
    r2 = set("asdfghjkl")
    r3 = set("zxcvbnm")

    result = []
    for word in words:
        if not word:
            continue

        # Determine which row the first character belongs
        first_char = word[0].lower()
        if first_char in r1:
            target_row = r1
        elif first_char in r2:
            target_row = r2
        else:
            target_row = r3

        # Check if all characters in the word belong to th
        if all(c.lower() in target_row for c in word):
            result.append(word)

    # Print results or "No words" if empty
    if not result:
        print("No words")
    else:
        for w in result:
            print(w)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	4 Hello Alaska Dad Peace	Alaska Dad	Alaska Dad	✓
✓	1 omk	No words	No words	✓
✓	2 adsfd afd	adsfd afd	adsfd afd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

You are given an integer tuple `nums` containing distinct numbers. Your task is to perform a sequence of operations on this tuple until it becomes empty. The operations are defined as follows:

1. If the first element of the tuple has the smallest value in the entire tuple, remove it.
2. Otherwise, move the first element to the end of the tuple.

You need to return an integer denoting the number of operations required to make the tuple empty.

Constraints

- The input tuple `nums` contains distinct integers.
- The operations must be performed using tuples and sets to maintain immutability and efficiency.
- Your function should accept the tuple `nums` as input and return the total number of operations as an integer.

Example:

Input: nums = (3, 4, -1)
Output: 5

Explanation:
Operation 1: [3, 4, -1] -> First element is not the smallest, move to the end -> [4, -1, 3]
Operation 2: [4, -1, 3] -> First element is not the smallest, move to the end -> [-1, 3, 4]
Operation 3: [-1, 3, 4] -> First element is the smallest, remove it -> [3, 4]
Operation 4: [3, 4] -> First element is the smallest, remove it -> [4]
Operation 5: [4] -> First element is the smallest, remove it -> []
Total operations: 5

For example:

Test	Result
`print(count_operations((3, 4, -1)))`	5

Answer: (penalty regime: 0 %)

Reset answer

```
def count_operations(nums: tuple) -> int:
    t = nums
    s = set(nums)  # Set to efficiently track and find the
    ops = 0

    while t:
        ops += 1
        # Check if the first element is the smallest in th
        if t[0] == min(s):
            removed = t[0]
            t = t[1:]       # Remove the first element usi
            s.remove(removed) # Remove from set
        else:
            # Move the first element to the end using tupl
            t = t[1:] + (t[0],)

    return ops
```

	Test	Expected	Got	
✓	`print(count_operations((3, 4, -1)))`	5	5	✓
✓	`print(count_operations((1, 2, 3, 4, 5)))`	5	5	✓
✓	`print(count_operations((5, 4, 3, 2, 1)))`	15	15	✓
✓	`print(count_operations((42, )))`	1	1	✓
✓	`print(count_operations((-2, 3, -5, 4, 1)))`	11	11	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a tuple and a positive integer k, the task is to find the count of distinct pairs in the tuple whose sum is equal to **K**.

Examples:

> **Input:** t = (5, 6, 5, 7, 7, 8), K = 13
> **Output:** 2
> **Explanation:**
> Pairs with sum K(= 13) are {(5, 8), (6, 7), (6, 7)}.
> Therefore, distinct pairs with sum K(= 13) are { (5, 8), (6, 7) }.
> Therefore, the required output is 2.

For example:

Input	Result
1,2,1,2,5 3	1
1,2 0	0

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all non-empty lines from standard input
    lines = [line.strip() for line in sys.stdin if line.

    i = 0
    while i + 1 < len(lines):
        # Parse the tuple from the comma-separated strir
        t = tuple(map(int, lines[i].split(',')))
        # Parse K from the next line
        k = int(lines[i+1])

        distinct_pairs = set()
        n = len(t)

        # Check all unique index pairs
        for idx1 in range(n):
            for idx2 in range(idx1 + 1, n):
                if t[idx1] + t[idx2] == k:
                    # Store sorted value tuple to ensure
                    pair = tuple(sorted((t[idx1], t[idx2
                    distinct_pairs.add(pair)

        print(len(distinct_pairs))
        i += 2

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	5,6,5,7,7,8 13	2	2	✓
✓	1,2,1,2,5 3	1	1	✓
✓	1,2 0	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Coders here is a simple task for you, Given string str. Your task is to check whether it is a binary string or not by using python set.

Examples:

Input: str = "01010101010"

Output: Yes

Input: str = "REC101"

Output: No

For example:

Input	Result
01010101010	Yes
010101 10101	No

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()

    for line in lines:
        if not line:
            continue

        # Convert the string into a set of its character
        char_set = set(line)

        # Check if the set of characters is a subset of
        if char_set.issubset({'0', '1'}):
            print("Yes")
        else:
            print("No")

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	01010101010	Yes	Yes	✓
✓	REC123	No	No	✓
✓	010101 10101	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

The **DNA sequence** is composed of a series of nucleotides abbreviated as `'A'`, `'C'`, `'G'`, and `'T'`.

- For example, `"ACGAATTCCG"` is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string `s` that represents a **DNA sequence**, return all the `10`**-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
Output: ["AAAAACCCCC","CCCCCAAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

For example:

Input	Result
AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCC CCCCCAAAAA

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()

    for s in lines:
        s = s.strip()
        if len(s) < 10:
            continue

        seen = set()
        repeated = []
        added = set()

        # Slide a window of length 10 across the string
        for i in range(len(s) - 9):
            sub = s[i:i+10]
            if sub in seen:
                if sub not in added:
                    repeated.append(sub)
                    added.add(sub)
            else:
                seen.add(sub)

        # Print each repeated sequence in order of appeara
        for r in repeated:
            print(r)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT	AAAAACCCCCCCCCCAAAAA	AAAAACCCCCCCCCCAAAAA	✓
✓	AAAAAAAAAAAAA	AAAAAAAAAA	AAAAAAAAAA	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums` containing `n + 1` integers where each integer is in the range `[1, n]` inclusive.There is only **one repeated number** in `nums`, return *this repeated number*. Solve the problem using set.

Example 1:

```
Input: nums = [1,3,4,2,2]
```

```
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
```

```
Output: 3
```

For example:

Input	Result
1 3 4 4 2	4

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    # Read all tokens from standard input dynamically
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    nums = [int(x) for x in input_data]

    seen = set()
    for num in nums:
        if num in seen:
            print(num)
            return
        seen.add(num)

if __name__ == '__main__':
    solve()
```

	Input	Expected	Got	
✓	1 3 4 4 2	4	4	✓
✓	1 2 2 3 4 5 6 7	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.